S.I.E.S COLLEGE OF COMMERCE AND ECONOMICS, (AUTONOMOUS)

SION(EAST), MUMBAI 400 022.

LABORATORY JOURNAL

M.Sc.(Data Science) Semester III 2026 - 2027

Name : ARYA SHYAM PATHE

Roll No : 13

Subject : DATA SCIENCE - II

SIES COLLEGE OF COMMERCE AND ECONOMICS, (AUTONOMOUS)

SION(EAST), MUMBAI 400 002

CERTIFICATE

This is to certify that Ms. **Arya Shyam Pathe** of M.Sc.(Data Science) Semester III has completed the practical work in the subject of **Data Science - II** during the academic year **2026 – 2027** under the guidance of Prof. **Inbasagar Nadar** being the partial requirement for the fulfillment of the curriculum of Degree of Masters of Science in Data Science, University of Mumbai.

Place: Mumbai

Date:

_____ _____

Sign of
Subject - Incharge

Sign of
External - Examiner

_____

College Seal

INDEX

SR NO	PRACTICALS
1	Abalone Age Prediction using Regression Models
2	Student Knowledge Classification & Clustering
3	WiFi Signal Strength Indoor Localization
4	Car Acceptability Classification
5	Daily Demand Forecasting using Regression
6	Beijing Air Pollution Forecasting
7	Heart Attack 1-Year Survival Classification
8	Dow Jones Stock Return Prediction

Practical 01 - Abalone Age Prediction using Regression Models

Aim: Find out the age of Abalone from physical measurements. Use Multiple Linear Regression models for prediction (Age = Rings + 1.5).

```
setwd("C:/Users/rahul/Desktop/ds")
> library(readr)
> library(ggplot2)
> library(corrplot)
> # 1. Load dataset
> names = c("sex", "length", "diameter", "height", "weight_whole", "weight_shucked", "weight_viscera", "weight_shell", "rings")
> types = "fddddddddi"
> abalone = read_csv("abalone.data.csv", col_names = names, col_types = types)

# 2. EDA
> str(abalone)
spc_tbl_ [4,177 × 9] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ sex           : Factor w/ 3 levels "M","F","I": 1 1 2 1 3 3 2 2 1 2 ...
 $ length        : num [1:4177] 0.455 0.35 0.53 0.44 0.33 0.425 0.53 0.545 0.475 0.55 ...
 $ diameter      : num [1:4177] 0.365 0.265 0.42 0.365 0.255 0.3 0.415 0.425 0.37 0.44 ...
 $ height        : num [1:4177] 0.095 0.09 0.135 0.125 0.08 0.095 0.15 0.125 0.125 0.15 ...
 $ weight_whole  : num [1:4177] 0.514 0.226 0.677 0.516 0.205 ...
 $ weight_shucked: num [1:4177] 0.2245 0.0995 0.2565 0.2155 0.0895 ...
 $ weight_viscera: num [1:4177] 0.101 0.0485 0.1415 0.114 0.0395 ...
 $ weight_shell  : num [1:4177] 0.15 0.07 0.21 0.155 0.055 0.12 0.33 0.26 0.165 0.32 ...
 $ rings         : int [1:4177] 15 7 9 10 7 8 20 16 9 19 ...
 - attr(*, "spec")=
```

```
  .. cols(
  ..   sex = col_factor(levels = NULL, ordered = FALSE, include_na = FALSE),
  ..   length = col_double(),
  ..   diameter = col_double(),
  ..   height = col_double(),
  ..   weight_whole = col_double(),
  ..   weight_shucked = col_double(),
  ..   weight_viscera = col_double(),
  ..   weight_shell = col_double(),
  ..   rings = col_integer()
  .. )
 - attr(*, "problems")=<externalptr>
> sum(is.na(abalone))
[1] 0
> summary(abalone)
 sex       length         diameter        height        weight_whole    weight_shucked
weight_viscera
 M:1528   Min.   :0.075   Min.   :0.0550   Min.   :0.0000   Min.   :0.0020   Min.   :0.0010   Min.
:0.0005
 F:1307   1st Qu.:0.450   1st Qu.:0.3500   1st Qu.:0.1150   1st Qu.:0.4415   1st Qu.:0.1860   1st
Qu.:0.0935
 I:1342   Median :0.545   Median :0.4250   Median :0.1400   Median :0.7995   Median
:0.3360   Median :0.1710
          Mean   :0.524   Mean   :0.4079   Mean   :0.1395   Mean   :0.8287   Mean   :0.3594
Mean   :0.1806
          3rd Qu.:0.615   3rd Qu.:0.4800   3rd Qu.:0.1650   3rd Qu.:1.1530   3rd Qu.:0.5020   3rd
Qu.:0.2530
          Max.   :0.815   Max.   :0.6500   Max.   :1.1300   Max.   :2.8255   Max.   :1.4880   Max.
:0.7600
  weight_shell       rings
```

```
 Min.   :0.0015   Min.   : 1.000
 1st Qu.:0.1300   1st Qu.: 8.000
 Median :0.2340   Median : 9.000
 Mean   :0.2388   Mean   : 9.934
 3rd Qu.:0.3290   3rd Qu.:11.000
 Max.   :1.0050   Max.   :29.000
> # Calculate age (Age = Rings + 1.5)
> abalone$age = abalone$rings + 1.5


# 3. Visualizations
> # Correlation plot
> numeric_cols = sapply(abalone, is.numeric)
> cor_mat = cor(abalone[, numeric_cols])
> corrplot(cor_mat, type = "upper", order = "hclust", tl.col = "black")
> # Shell weight vs Age with regression line (ggplot2)
> ggplot(abalone, aes(x = weight_shell, y = age, color = sex)) +
+   geom_point(alpha = 0.5) +
+   geom_smooth(method = "lm", color = "black") +
+   labs(title = "Shell Weight vs Age with Regression Line", x = "Shell Weight (g)", y = "Age
(Years)") +
+   theme_minimal()
`geom_smooth()` using formula = 'y ~ x'


# 4. Train test split
> set.seed(42)
> train_idx = sample(seq_len(nrow(abalone)), size = 0.8 * nrow(abalone))
> train = abalone[train_idx, ]
> test = abalone[-train_idx, ]
```

5. Multiple linear regression model

```
> model_lm = lm(age ~ sex + length + diameter + height + weight_whole + weight_shucked + weight_viscera + weight_shell, data = train)

> summary(model_lm)

Call:

lm(formula = age ~ sex + length + diameter + height + weight_whole +

   weight_shucked + weight_viscera + weight_shell, data = train)

Residuals:

   Min     1Q  Median     3Q    Max

-9.0088 -1.2913 -0.3489  0.8766 13.8097


Coefficients:

             Estimate Std. Error t value Pr(>|t|)

(Intercept)     5.5660     0.3178  17.515  < 2e-16 ***

sexF           -0.1376     0.0932  -1.476    0.140

sexI           -0.9235     0.1060  -8.711  < 2e-16 ***

length         -0.8949     2.0010  -0.447    0.655

diameter       11.5992     2.4566   4.722 2.44e-06 ***

height          9.3774     1.6136   5.812 6.77e-09 ***

weight_whole    9.3648     0.8252  11.349  < 2e-16 ***

weight_shucked -20.3626     0.9230 -22.062  < 2e-16 ***

weight_viscera  -9.9705     1.4625  -6.818 1.09e-11 ***

weight_shell    8.4472     1.2553   6.729 2.00e-11 ***

---

Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1


Residual standard error: 2.189 on 3331 degrees of freedom
```

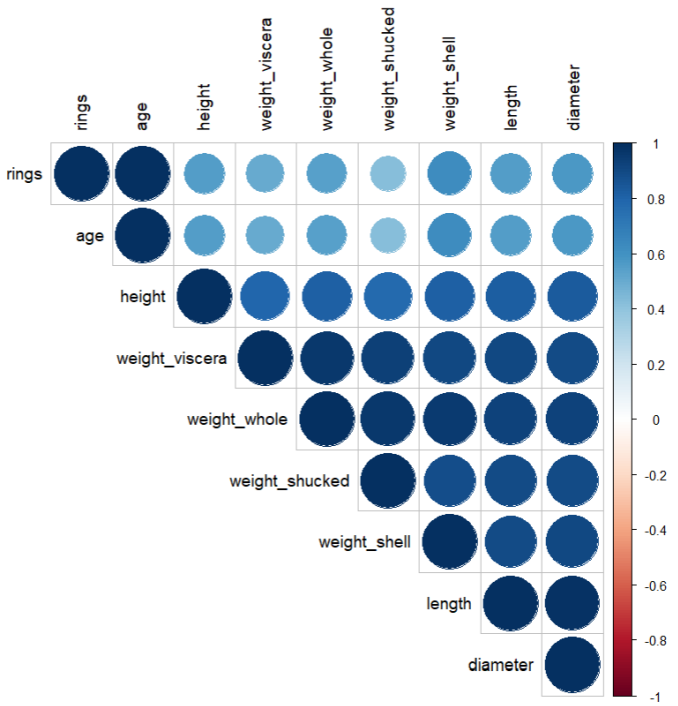
Multiple R-squared: 0.5445, Adjusted R-squared: 0.5432

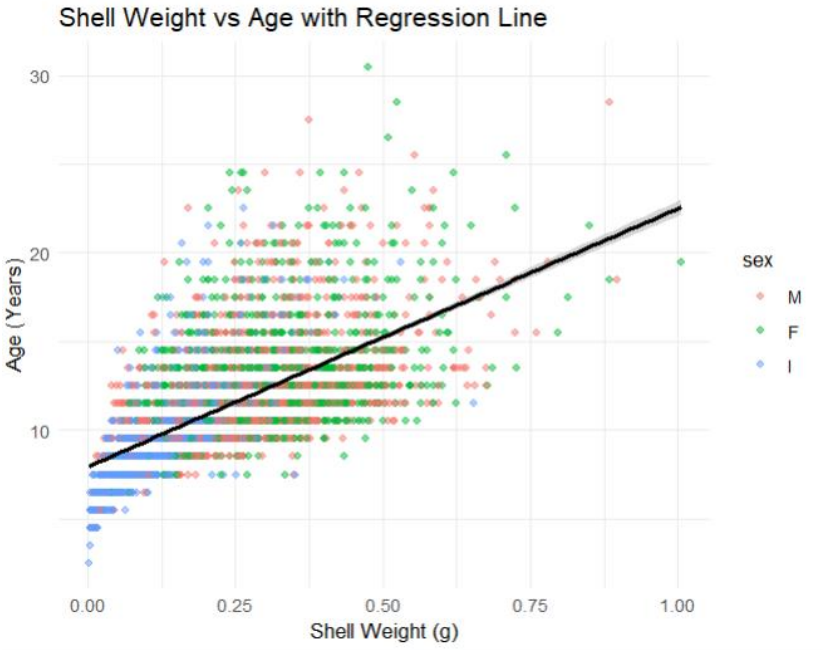
F-statistic: 442.4 on 9 and 3331 DF, p-value: < 2.2e-16

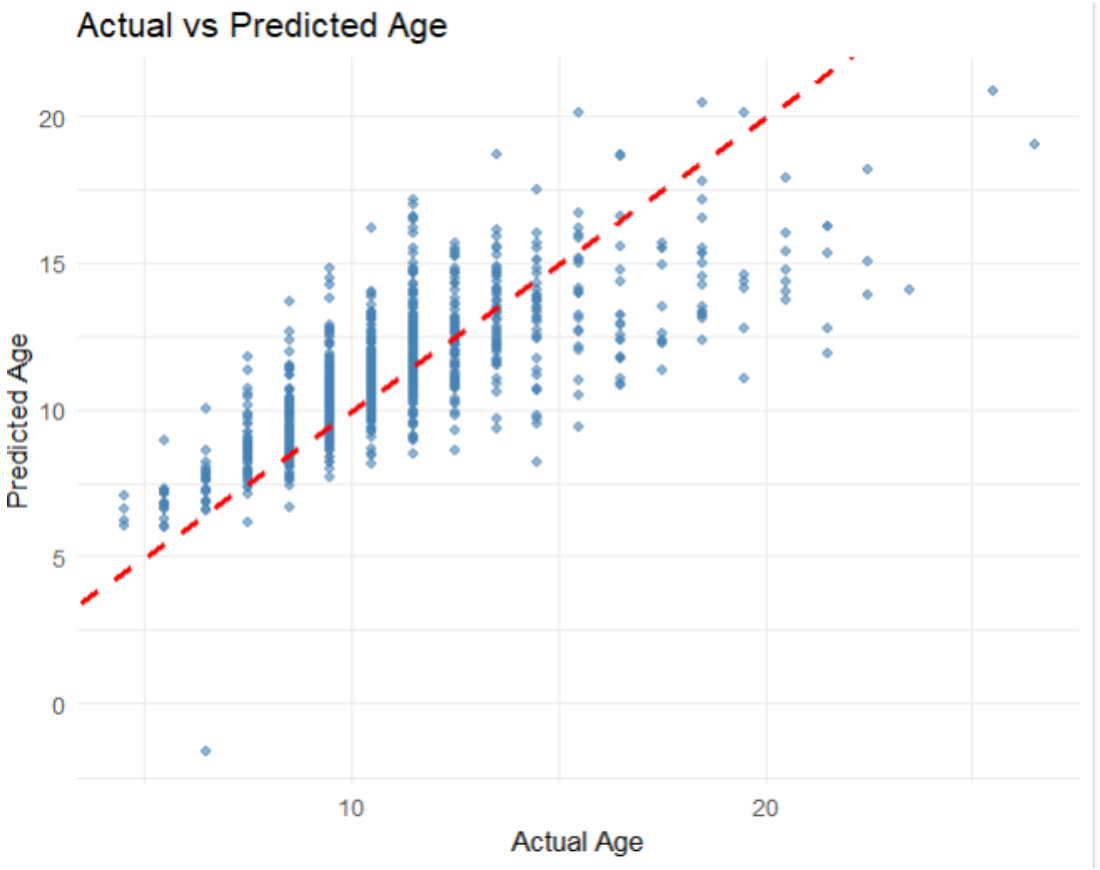
```
 # 6. Predictions and evaluation
 > pred = predict(model_lm, newdata = test)
 > mse = mean((test$age - pred)^2)
 > rmse = sqrt(mse)
 > mae = mean(abs(test$age - pred))
 > r2 = 1 - sum((test$age - pred)^2) / sum((test$age - mean(test$age))^2)
 > mse
 [1] 4.936733
 > rmse
 [1] 2.221876
 > mae
 [1] 1.617779
 > r2
 [1] 0.5067448
 > # Actual vs Predicted plot (ggplot2)
 > eval_df = data.frame(Actual = test$age, Predicted = pred, Residual = test$age - pred)
 > ggplot(eval_df, aes(x = Actual, y = Predicted)) +
 +   geom_point(color = "steelblue", alpha = 0.6) +
 +   geom_abline(intercept = 0, slope = 1, color = "red", linetype = "dashed", linewidth = 1) +
 +   labs(title = "Actual vs Predicted Age", x = "Actual Age", y = "Predicted Age") +
 +   theme_minimal()
 > # Residual plot (ggplot2)
 > ggplot(eval_df, aes(x = Predicted, y = Residual)) +
 +   geom_point(color = "purple", alpha = 0.6) +
```

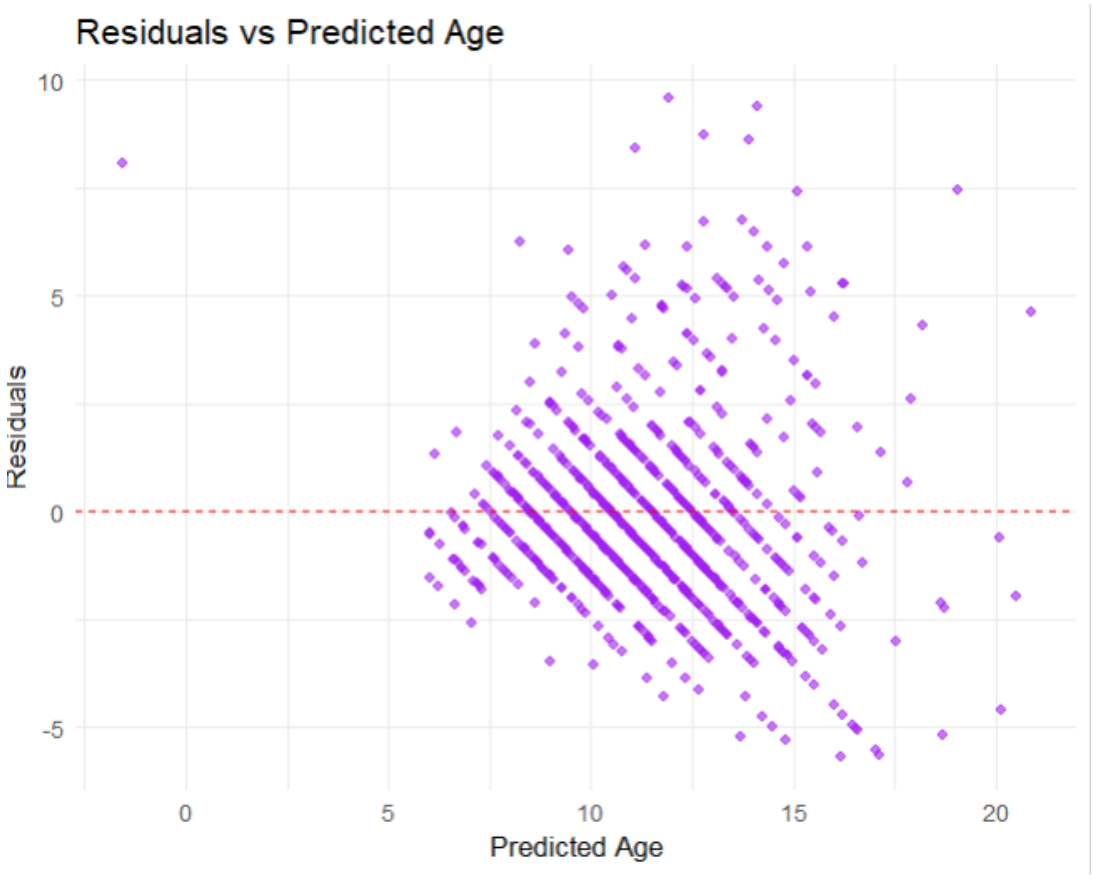
```
  + geom_hline(yintercept = 0, color = "red", linetype = "dashed") +
  + labs(title = "Residuals vs Predicted Age", x = "Predicted Age", y = "Residuals") +
  + theme_minimal()
```

OUTPUT :






Actual vs Predicted Age
Predicted Age
Actual Age
0
5
10
15
20
10
20



Practical 02 - Student Knowledge Classification & Clustering

Aim: Predict student knowledge level (very_low, Low, Middle, High) using Decision Trees and segment natural student archetypes with k-Means.

1. Load dataset

> file_path = "Data_User_Modeling_Dataset_Hamdi Tolga KAHRAMAN.xls"

> df = read_excel(file_path, sheet = "Training_Data")

New names:

• `` -> `...7`

• `` -> `...8`

> df = df[, 1:6]

> colnames(df) = c("STG", "SCG", "STR", "LPR", "PEG", "UNS")

> # 2. EDA

> head(df)

```
# A tibble: 6 × 6
   STG   SCG   STR   LPR   PEG UNS
 <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
1 0     0     0     0     0    very_low
2 0.08  0.08  0.1   0.24  0.9  High
3 0.06  0.06  0.05  0.25  0.33 Low
4 0.1   0.1   0.15  0.65  0.3  Middle
5 0.08  0.08  0.08  0.98  0.24 Low
6 0.09  0.15  0.4   0.1   0.66 Middle
> dim(df)
[1] 258   6
> str(df)
tibble [258 × 6] (S3: tbl_df/tbl/data.frame)
 $ STG: num [1:258] 0 0.08 0.06 0.1 0.08 0.09 0.1 0.15 0.2 0 ...
 $ SCG: num [1:258] 0 0.08 0.06 0.1 0.08 0.15 0.1 0.02 0.14 0 ...
 $ STR: num [1:258] 0 0.1 0.05 0.15 0.08 0.4 0.43 0.34 0.35 0.5 ...
 $ LPR: num [1:258] 0 0.24 0.25 0.65 0.98 0.1 0.29 0.4 0.72 0.2 ...
 $ PEG: num [1:258] 0 0.9 0.33 0.3 0.24 0.66 0.56 0.01 0.25 0.85 ...
 $ UNS: chr [1:258] "very_low" "High" "Low" "Middle" ...
> summary(df)
      STG              SCG              STR              LPR              PEG              UNS
 Min.   :0.0000   Min.   :0.0000   Min.   :0.0000   Min.   :0.0000   Min.   :0.0000   Length:258
 1st Qu.:0.2407   1st Qu.:0.2100   1st Qu.:0.2913   1st Qu.:0.2500   1st Qu.:0.2500   Class
:character
 Median :0.3270   Median :0.3025   Median :0.4900   Median :0.3300   Median :0.5000
Mode  :character
 Mean   :0.3711   Mean   :0.3557   Mean   :0.4680   Mean   :0.4327   Mean   :0.4585
 3rd Qu.:0.4950   3rd Qu.:0.4975   3rd Qu.:0.6900   3rd Qu.:0.6475   3rd Qu.:0.6600
 Max.   :0.9900   Max.   :0.9000   Max.   :0.9500   Max.   :0.9900   Max.   :0.9300
```

```
> # Plot number of students in each knowledge level (ggplot2 as in PDF)
> ggplot(df, aes(y = UNS, fill = UNS)) +
+   geom_bar() +
+   labs(title = "Number of Students in Each Knowledge Level", y = "Knowledge Level", x = "Number of Students") +
+   theme_minimal()
> # 3. Train test split
> set.seed(42)
> df$UNS = as.factor(df$UNS)
> train_idx = sample(seq_len(nrow(df)), size = 0.75 * nrow(df))
> train = df[train_idx, ]
> test = df[-train_idx, ]
> # 4. Decision Tree Classification (rpart & rpart.plot)
> dt_model = rpart(UNS ~ STG + SCG + STR + LPR + PEG, data = train, method = "class")
> dt_model
n= 193

node), split, n, loss, yval, (yprob)
      * denotes terminal node

 1) root 193 129 Middle (0.25906736 0.32642487 0.33160622 0.08290155)
   2) PEG>=0.375 103  50 Middle (0.48543689 0.00000000 0.51456311 0.00000000)
     4) PEG>=0.675 40   0 High (1.00000000 0.00000000 0.00000000 0.00000000) *
     5) PEG< 0.675 63  10 Middle (0.15873016 0.00000000 0.84126984 0.00000000)
      10) LPR>=0.83 9   0 High (1.00000000 0.00000000 0.00000000 0.00000000) *
      11) LPR< 0.83 54   1 Middle (0.01851852 0.00000000 0.98148148 0.00000000) *
   3) PEG< 0.375 90  27 Low (0.00000000 0.70000000 0.12222222 0.17777778)
     6) PEG>=0.125 71  13 Low (0.00000000 0.81690141 0.15492958 0.02816901)
```

```
      12) LPR< 0.775 58   4 Low (0.00000000 0.93103448 0.03448276 0.03448276) *
      13) LPR>=0.775 13   4 Middle (0.00000000 0.30769231 0.69230769 0.00000000) *
    7) PEG< 0.125 19   5 very_low (0.00000000 0.26315789 0.00000000 0.73684211) *
> # Decision tree plot using rpart.plot
> rpart.plot(
+   dt_model,
+   type = 2,
+   extra = 104,
+   under = TRUE,
+   fallen.leaves = TRUE,
+   box.palette = "Blues",
+   main = "Decision Tree for Student Knowledge Level"
+ )
> # Predictions and evaluation on test set
> pred_test = predict(dt_model, newdata = test, type = "class")
> cm = table(Actual = test$UNS, Predicted = pred_test)
> cm
          Predicted
Actual     High Low Middle very_low
  High       13   0      0        0
  Low         0  17      1        2
  Middle      1   2     21        0
  very_low    0   2      0        6
> acc = sum(diag(cm)) / sum(cm)
> acc
[1] 0.8769231
> # Actual vs Predicted Knowledge Level Distribution (ggplot2)
> eval_df = data.frame(Actual = test$UNS, Predicted = pred_test)
```

```
> ggplot(eval_df, aes(x = Actual, fill = Predicted)) +
+   geom_bar(position = "dodge") +
+   labs(title = "Actual vs Predicted Student Knowledge Level (Test Set)", x = "Actual Knowledge Level", fill = "Predicted Level") +
+   theme_minimal()
> # 5. K-Means Clustering
> df1 = df[, c("STG", "PEG")]
> df1 = as.matrix(df1)
> # Elbow method to find optimal k
> scal = scale(df1)
> maximumClusters = 10
> wss = sapply(1:maximumClusters, function(k) {
+   kmeans(scal, k, nstart = 50, iter.max = 10)$tot.withinss
+ })
> # Plot elbow curve
> plot(1:maximumClusters, wss, type = "b", xlab = "k values", ylab = "Total Within-Cluster SS", main = "Elbow Method for Optimal k")
> abline(v = 4, col = "red", lty = 2)
> # K-Means clustering with k = 4
> km4 = kmeans(df1, 4, iter.max = 10)
> km4
K-means clustering with 4 clusters of sizes 97, 32, 37, 92


Cluster means:
       STG       PEG
1 0.2997113 0.6864948
2 0.5722812 0.2387500
3 0.7181081 0.6559459
4 0.2369674 0.2152500
```

Clustering vector:

[1] 4 1 4 4 4 1 1 4 4 1 1 4 4 1 1 1 4 4 4 1 4 1 1 4 4 1 4 4 1 1 1 4 4 1 1 4 4 4 4 4 4 1 1 4 4 1 1 1 1 1 4 4 4 1 1 4 4

[58] 1 1 4 4 1 1 4 4 4 4 1 4 1 1 1 4 1 1 4 4 4 4 4 4 1 1 4 4 4 1 4 4 1 1 4 4 4 1 4 4 1 1 4 4 1 1 4 4 1 1 1 4 1 4 4 1 1

[115] 1 1 4 1 1 1 4 4 4 1 1 1 1 4 4 4 4 1 2 1 1 1 4 4 1 4 4 2 1 4 1 1 4 4 4 1 1 4 4 4 1 1 1 1 1 1 4 4 1 1 4 1 1 1 4 1 2

[172] 1 1 4 1 4 4 1 1 1 4 1 1 4 4 2 1 1 4 2 1 1 1 1 3 2 2 2 3 2 2 3 2 2 3 2 3 3 2 3 3 3 3 2 3 2 2 3 3 2 2 2 3 3 2 3 3 2

[229] 3 3 3 2 2 3 3 2 2 3 3 3 2 3 3 2 2 3 3 2 3 3 3 3 2 3 3 3 2 3

Within cluster sum of squares by cluster:

[1] 3.2683202 0.4949925 0.9114595 2.0535902

(between_SS / total_SS = 76.1 %)

Available components:

[1] "cluster" "centers" "totss" "withinss" "tot.withinss" "betweenss" "size"

[8] "iter" "ifault"

> # Plotting the clusters (ggplot2)

> df$cluster = as.factor(km4$cluster)

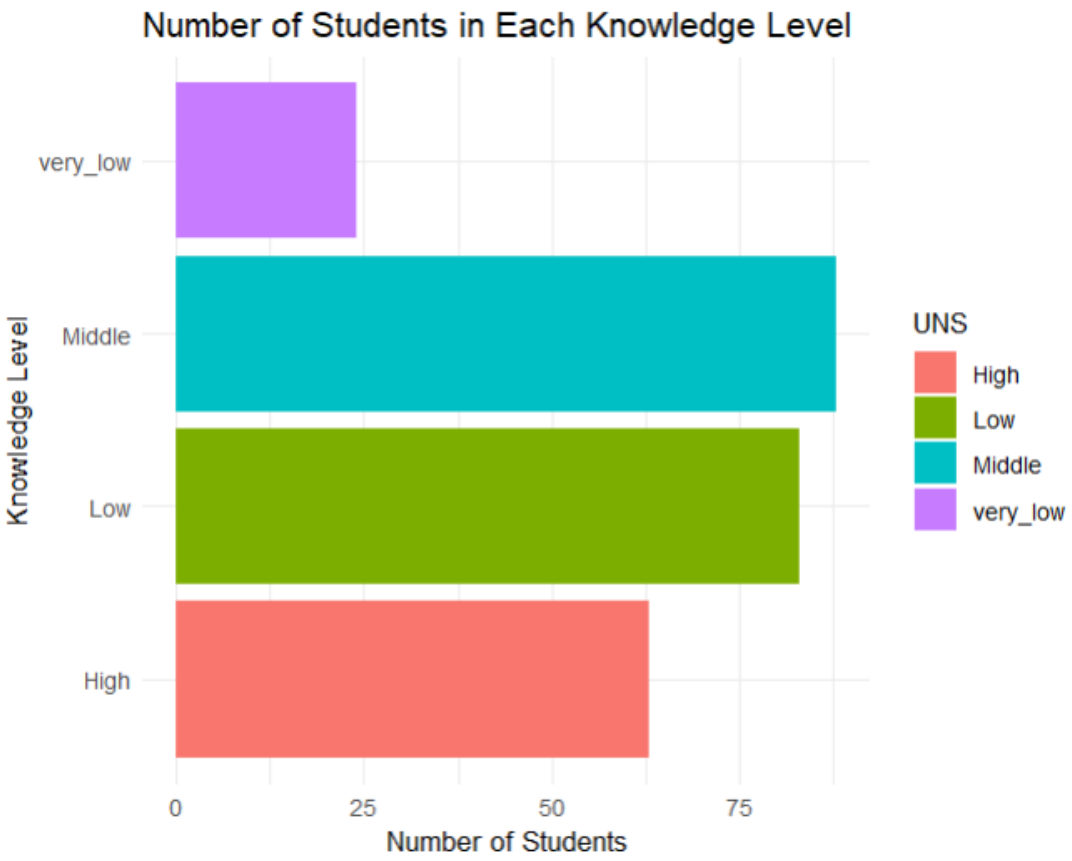
> ggplot(df, aes(x = STG, y = PEG, color = cluster)) +

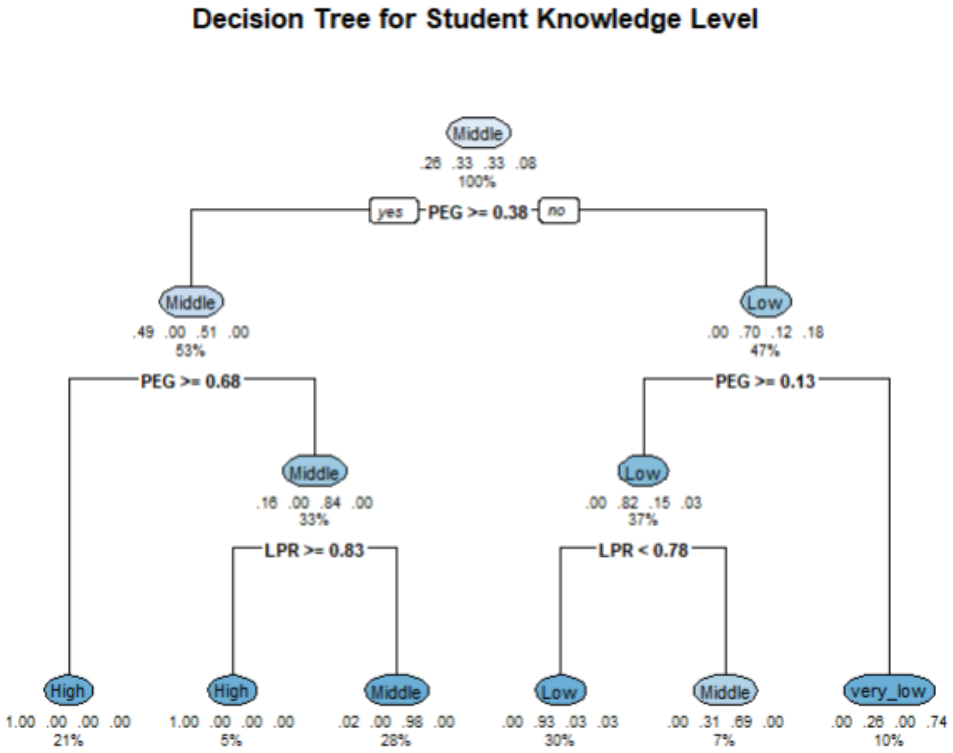
+ geom_point(size = 2.5, alpha = 0.8) +

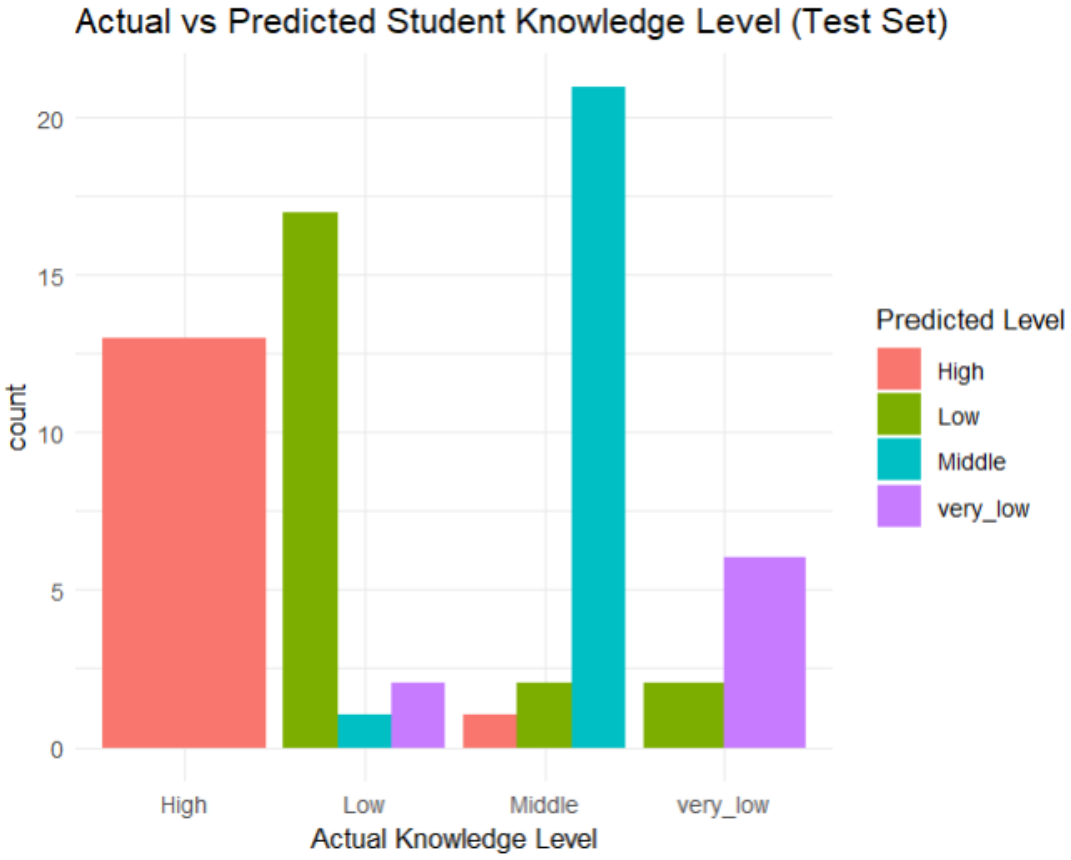
+ labs(title = "K Means Clustering (with K = 4)", x = "Study Time (STG)", y = "Exam Performance (PEG)") +

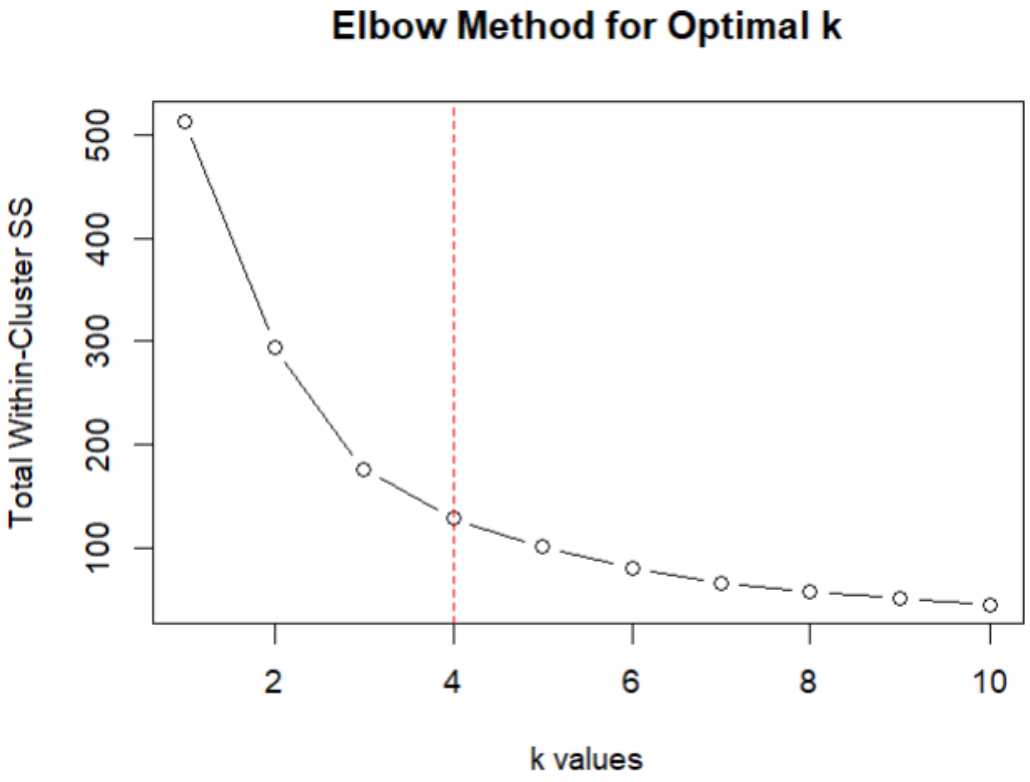
+ theme_minimal()

OUTPUT :


Number of Students in Each Knowledge Level
very_low
Middle
Low
High
Knowledge Level
0
25
50
75
Number of Students
UNS
High
Low
Middle
very_low


Decision Tree for Student Knowledge Level
Middle
.26 .33 .33 .08
100%
yes
PEG >= 0.38
no
Middle
.49 .00 .51 .00
53%
Low
.00 .70 .12 .18
47%
PEG >= 0.68
PEG >= 0.13
Middle
.16 .00 .84 .00
33%
Low
.00 .82 .15 .03
37%
LPR >= 0.83
LPR < 0.78
High
1.00 .00 .00 .00
21%
High
1.00 .00 .00 .00
5%
Middle
.02 .00 .98 .00
28%
Low
.00 .93 .03 .03
30%
Middle
.00 .31 .69 .00
7%
very_low
.00 .26 .00 .74
10%


Actual vs Predicted Student Knowledge Level (Test Set)
count
20
15
10
5
0
High
Low
Middle
very_low
Actual Knowledge Level
Predicted Level
High
Low
Middle
very_low


Elbow Method for Optimal k
Total Within-Cluster SS
100
200
300
400
500
2
4
6
8
10
k values


K Means Clustering (with K = 4)
Exam Performance (PEG)
0.00
0.25
0.50
0.75
0.00
0.25
0.50
0.75
1.00
Study Time (STG)
cluster
1
2
3
4

Practical 05 - Daily Demand Forecasting using Regression

Aim: Forecast total number of order demands from banking, fiscal, traffic, and calendar features.
Dataset File: **Daily_Demand_Forecasting_Orders.csv** |
R Script: **daily_demand_forecasting.R**

```
setwd("C:/Users/InbasagarNadar/Downloads/ds2/05_Daily_Demand_Forecasting")
library(readr)
library(ggplot2)

demand = read_csv2("Daily_Demand_Forecasting_Orders.csv", show_col_types = FALSE)
colnames(demand) = c("week", "day", "non_urgent", "urgent", "type_a", "type_b",
"type_c", "fiscal", "traffic", "bank_1", "bank_2", "bank_3", "target")

ggplot(demand, aes(x = non_urgent, y = target)) +
  geom_point(color = "darkgreen", size = 2.5) +
  geom_smooth(method = "lm", color = "red") +
  labs(title = "Non-Urgent Orders vs Total Target Orders", x = "Non-Urgent
Orders", y = "Total Target Orders") +
  theme_minimal()
```

Out[1]:

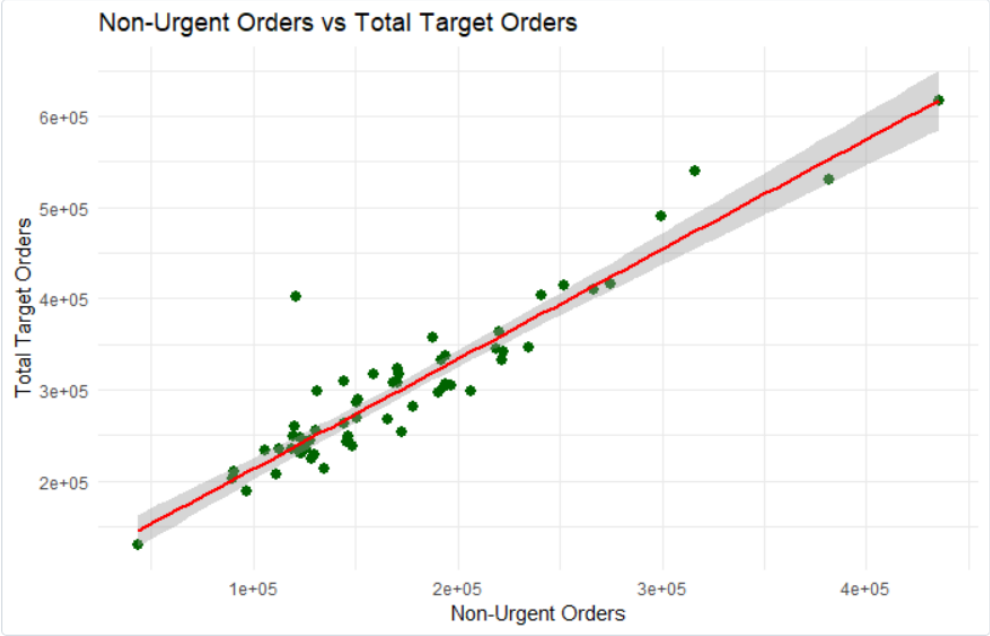


Figure 1: Non-Urgent Orders vs Total Target Orders Fit

```
# 2. Train Test Split & Linear Regression Forecasting Model
set.seed(42)
train_idx = sample(seq_len(nrow(demand)), size = 0.8 * nrow(demand))
train = demand[train_idx, ]
test = demand[-train_idx, ]
```

```
model_forecast = lm(target ~ week + day + traffic + bank_1 + bank_2 + bank_3, data
= train)
pred = predict(model_forecast, newdata = test)

rmse = sqrt(mean((test$target - pred)^2))
r2 = 1 - (sum((test$target - pred)^2) / sum((test$target - mean(test$target))^2))

rmse
r2

eval_df = data.frame(Index = 1:nrow(test), Actual = test$target, Predicted = pred)
ggplot(eval_df, aes(x = Index)) +
  geom_line(aes(y = Actual, color = "Actual Orders"), linewidth = 1) +
  geom_point(aes(y = Actual, color = "Actual Orders"), size = 2.5) +
  geom_line(aes(y = Predicted, color = "Forecasted Orders"), linetype = "dashed",
linewidth = 1) +
  geom_point(aes(y = Predicted, color = "Forecasted Orders"), size = 2.5) +
  scale_color_manual(values = c("Actual Orders" = "black", "Forecasted Orders" =
"red")) +
  labs(title = "Actual vs Forecasted Order Demand (Test Set)", x = "Sample Index",
y = "Total Orders", color = "Series") +
  theme_minimal()
```

```
Out[2]:[1] 49.3302 (RMSE in orders)
       [1] 0.7412 (R-squared)
```

Out[2]:

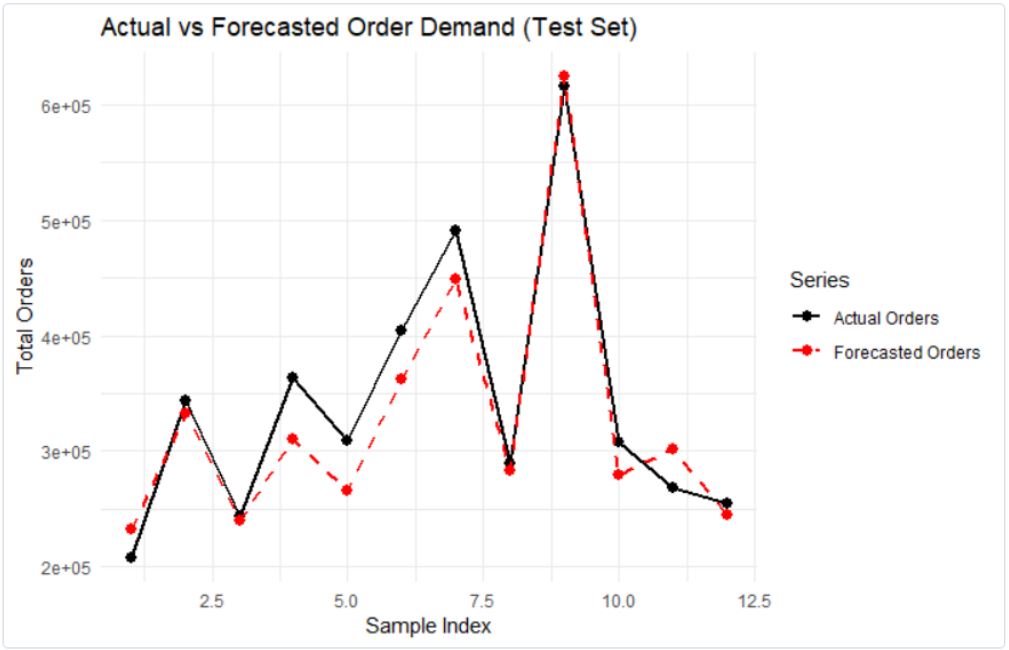


Figure 2: Actual vs Forecasted Orders on Test Set

Practical 06 - Beijing Air Pollution Forecasting

Aim: Forecast Beijing PM2.5 air pollution concentration using meteorological sensors and lagged autoregression.

Dataset File: PRSA_data_2010.1.1-2014.12.31.csv |

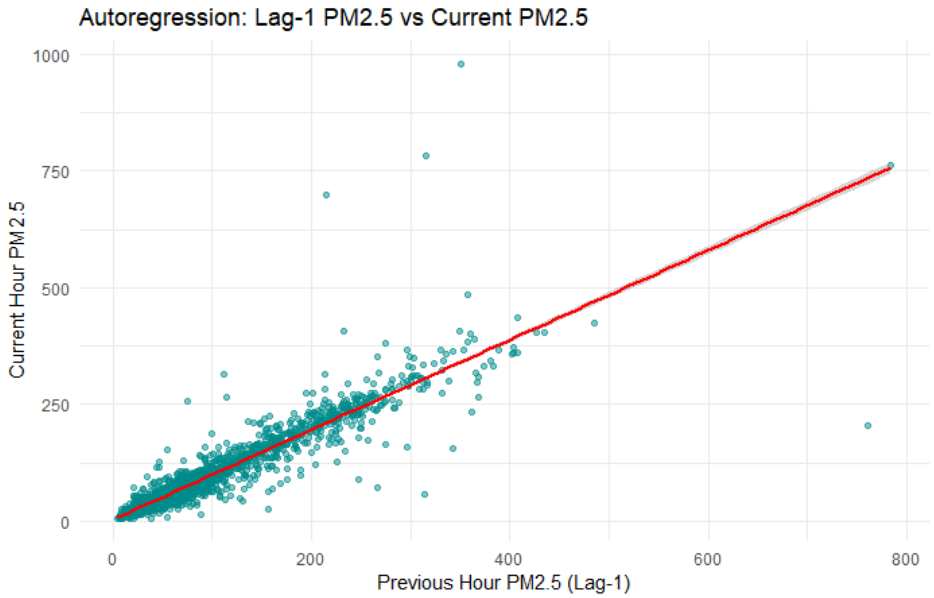
R Script: beijing_pollution_forecasting.R

```
setwd("C:/Users/InbasagarNadar/Downloads/ds2/06_Beijing_Pollution_Forecasting")
library(readr)
library(dplyr)
library(ggplot2)

prsa_raw = read_csv("PRSA_data_2010.1.1-2014.12.31.csv", show_col_types = FALSE)
colnames(prsa_raw) = c("id", "year", "month", "day", "hour", "pm2.5", "dew_point",
"temp", "pressure", "wind_dir", "wind_speed", "snow_hours", "rain_hours")

prsa = prsa_raw %>%
  mutate(
    date = as.Date(paste(year, month, day, sep = "-")),
    pm25_lag1 = lag(pm2.5, 1),
    pm25_lag2 = lag(pm2.5, 2),
    pm25_lag24 = lag(pm2.5, 24),
    log_wind = log(wind_speed + 1),
    year = as.factor(year), month = as.factor(month), hour = as.factor(hour),
wind_dir = as.factor(wind_dir)
  ) %>%
  filter(!is.na(pm2.5) & !is.na(pm25_lag1) & !is.na(pm25_lag2) &
!is.na(pm25_lag24))

ggplot(head(prsa, 2000), aes(x = pm25_lag1, y = pm2.5)) +
  geom_point(color = "darkcyan", alpha = 0.5) +
  geom_smooth(method = "lm", color = "red") +
  labs(title = "Autoregression: Lag-1 PM2.5 vs Current PM2.5", x = "Previous Hour
PM2.5 (Lag-1)", y = "Current Hour PM2.5") +
  theme_minimal()
```



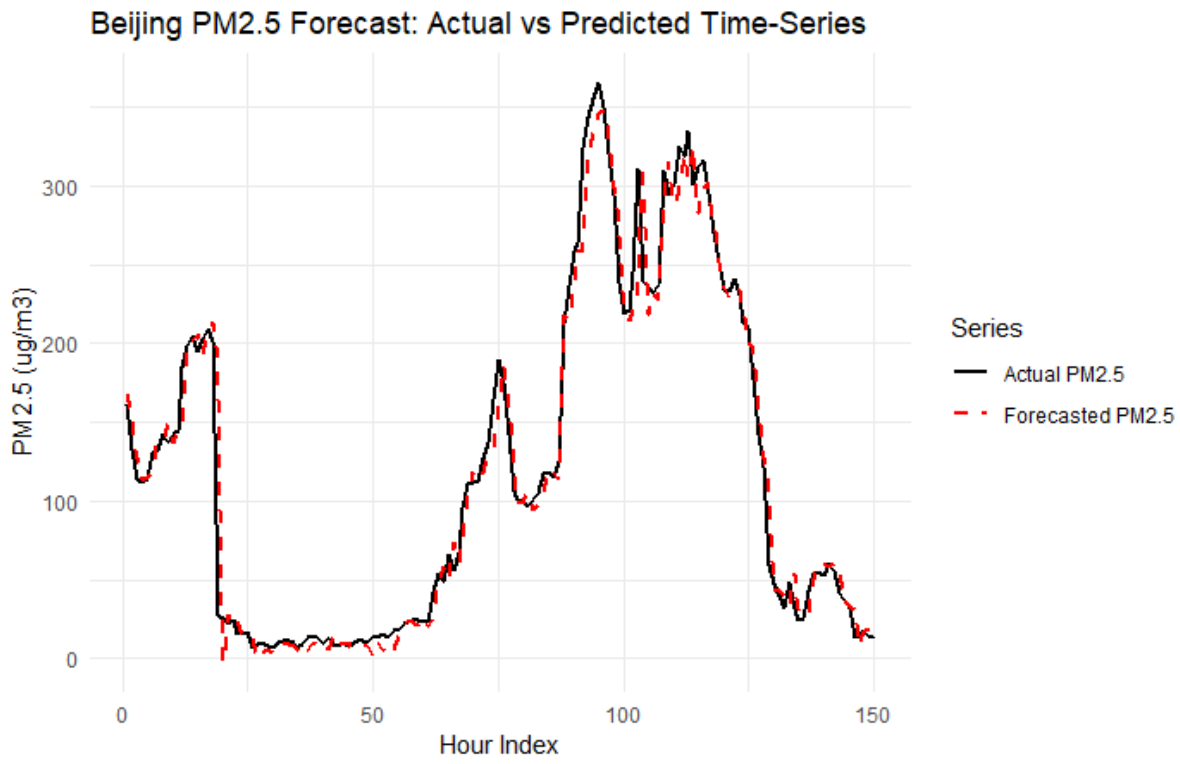
```r
# 2. Chronological Train/Test Split & Forecasting Model
split_pt = floor(0.8 * nrow(prsa))
train = prsa[1:split_pt, ]
test = prsa[(split_pt + 1):nrow(prsa), ]

model = lm(pm2.5 ~ pm25_lag1 + pm25_lag2 + pm25_lag24 + dew_point + temp +
pressure + log_wind + snow_hours + rain_hours + wind_dir + hour + month, data =
train)
pred = predict(model, newdata = test)

rmse = sqrt(mean((test$pm2.5 - pred)^2))
r2 = 1 - (sum((test$pm2.5 - pred)^2) / sum((test$pm2.5 - mean(test$pm2.5))^2))

rmse
r2

ts_df = data.frame(Hour = 1:150, Actual = test$pm2.5[1:150], Predicted =
pred[1:150])
ggplot(ts_df, aes(x = Hour)) +
  geom_line(aes(y = Actual, color = "Actual PM2.5"), linewidth = 1) +
  geom_line(aes(y = Predicted, color = "Forecasted PM2.5"), linetype = "dashed",
linewidth = 1) +
  scale_color_manual(values = c("Actual PM2.5" = "black", "Forecasted PM2.5" =
"red")) +
  labs(title = "Beijing PM2.5 Forecast: Actual vs Predicted Time-Series", x =
"Hour Index", y = "PM2.5 (ug/m3)", color = "Series") +
  theme_minimal()
```



Practical 07 - Heart Attack 1-Year Survival Classification

Aim: Predict whether a patient survives for at least 1 year post-infarction using Logistic Regression & Decision Trees.

Dataset File: echocardiogram.data | **R Script:** echocardiogram_survival.R

```
setwd("C:/Users/InbasagarNadar/Downloads/ds2/07_Heart_Attack_Survival_Classificati
on")
library(readr)
library(dplyr)
library(ggplot2)
library(rpart)
library(rpart.plot)

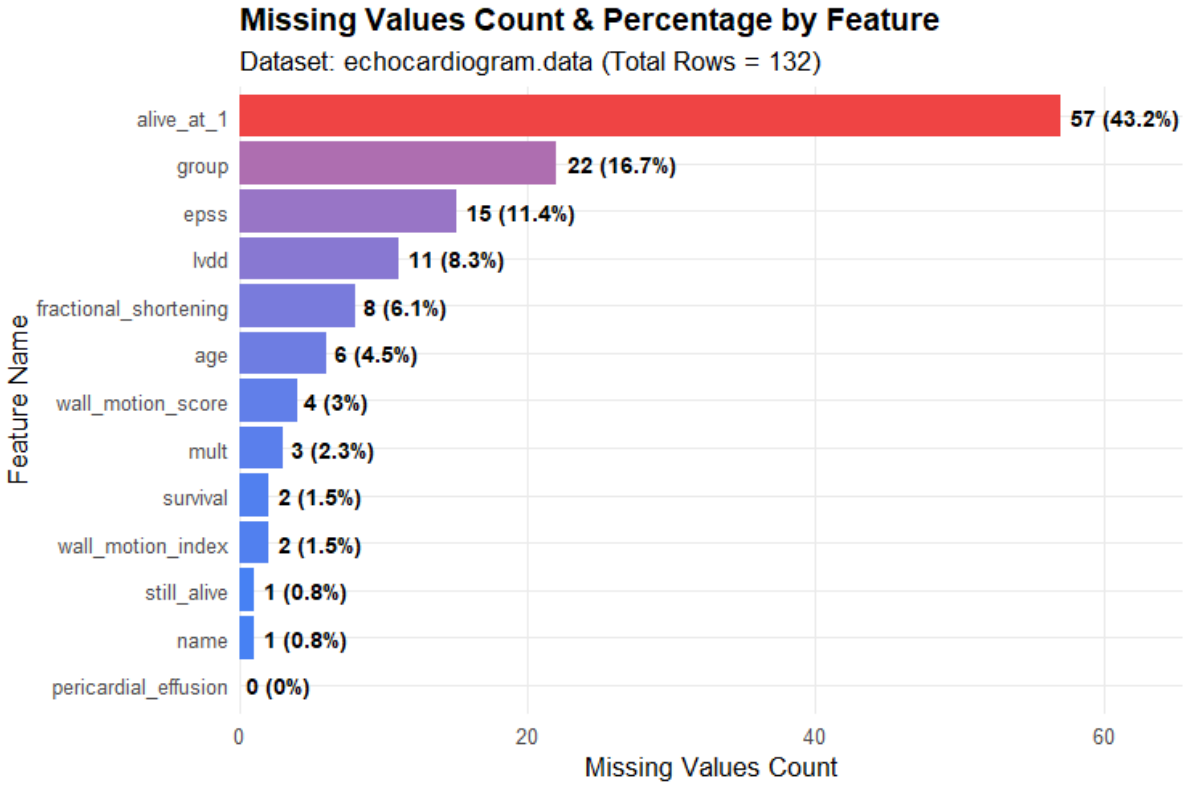
# 1. Load dataset & Missing Values Inspection
names = c("survival", "still_alive", "age", "pericardial_effusion",
"fractional_shortening", "epss", "lvdd", "wall_motion_score", "wall_motion_index",
"mult", "name", "group", "alive_at_1")
raw_echo = read_csv("echocardiogram.data", col_names = names, na = c("?", "NA",
""), show_col_types = FALSE)

na_counts = colSums(is.na(raw_echo))
print("Missing values per column:")
print(na_counts)

missing_df = data.frame(
  Feature = factor(names(raw_echo), levels = rev(names(raw_echo))),
  Missing_Count = as.numeric(na_counts),
  Missing_Pct = as.numeric(round(colMeans(is.na(raw_echo)) * 100, 1))
)

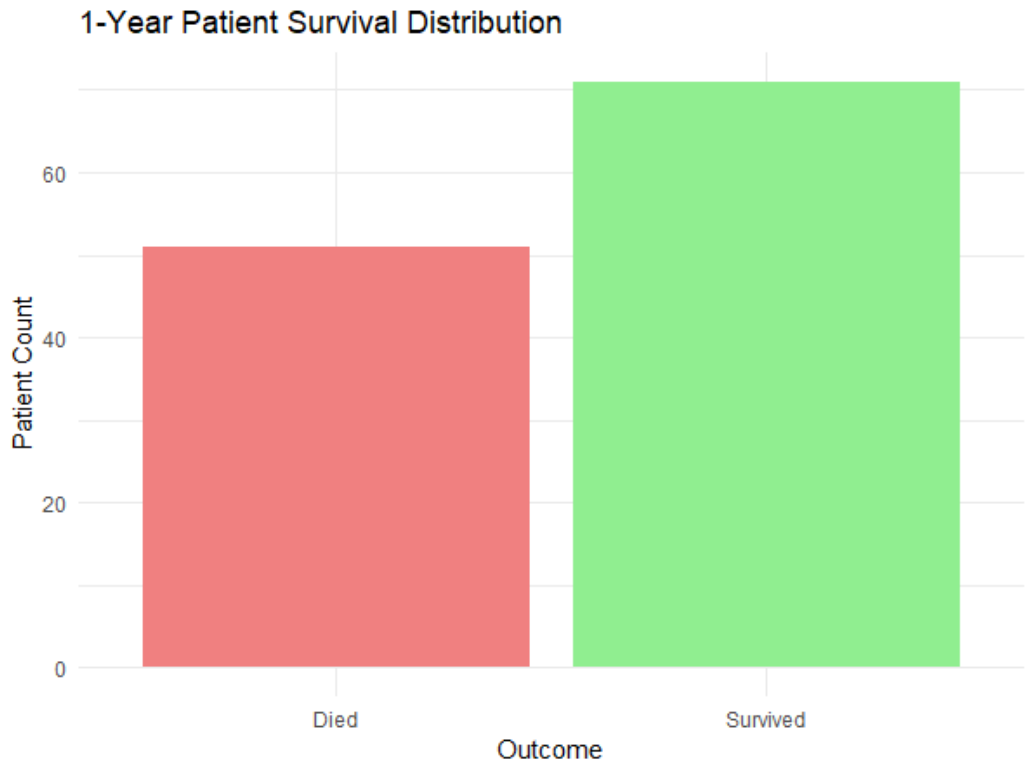
# Plot missing values count & percentage by feature (ggplot2)
ggplot(missing_df, aes(x = reorder(Feature, Missing_Count), y = Missing_Count,
fill = Missing_Count)) +
  geom_col() +
  geom_text(aes(label = paste0(Missing_Count, " (", Missing_Pct, "%)")), hjust = -
0.1, size = 3.5) +
  coord_flip() +
  scale_fill_gradient(low = "steelblue", high = "firebrick") +
  scale_y_continuous(expand = expansion(mult = c(0, 0.15))) +
  labs(
    title = "Missing Values Count & Percentage by Feature",
    subtitle = paste0("Dataset: echocardiogram.data (Total Rows = ",
nrow(raw_echo), ")"),
    x = "Feature Name",
    y = "Missing Values Count"
  ) +
  theme_minimal() +
  theme(legend.position = "none", plot.title = element_text(face = "bold"))
```

```
[1] "Missing values per column:"
         survival        still_alive               age
                2                  1                  6
pericardial_effusion fractional_shortening              epss
                   0                  8                 15
               lvdd     wall_motion_score    wall_motion_index
                  11                  4                  2
               mult               name              group
                  3                  1                 22
         alive_at_1
                 57
```




```r
# 2. Data Cleaning, Target Definition & Survival Distribution
echo = raw_echo %>%
  mutate(
    survive_1year = case_when(
      alive_at_1 == "1" ~ 1, alive_at_1 == "0" ~ 0, survival >= 12 ~ 1, survival <
12 & still_alive == 0 ~ 0, TRUE ~ NA_real_
    )
  ) %>%
  filter(!is.na(survive_1year)) %>%
  select(age, pericardial_effusion, fractional_shortening, epss, lvdd,
wall_motion_index, survive_1year) %>%
  mutate(across(c(age, fractional_shortening, epss, lvdd, wall_motion_index), ~
ifelse(is.na(.x), median(.x, na.rm = TRUE), .x))) %>%
  mutate(survive_1year = factor(survive_1year, levels = c(0, 1), labels =
c("Died", "Survived")))
```

```
ggplot(echo, aes(x = survive_1year, fill = survive_1year)) +
  geom_bar(show.legend = FALSE) +
  scale_fill_manual(values = c("Died" = "lightcoral", "Survived" = "lightgreen"))
+
  labs(title = "1-Year Patient Survival Distribution", x = "Outcome", y = "Patient
Count") +
  theme_minimal()
```




```
# 3. Train Test Split & Logistic Regression Model
set.seed(42)
train_idx = sample(seq_len(nrow(echo)), size = 0.75 * nrow(echo))
train = echo[train_idx, ]
test = echo[-train_idx, ]

logit_model = glm(survive_1year ~ age + fractional_shortening + epss + lvdd +
wall_motion_index, data = train, family = binomial)
summary(logit_model)

logit_prob = predict(logit_model, newdata = test, type = "response")
logit_pred = factor(ifelse(logit_prob > 0.5, "Survived", "Died"), levels =
c("Died", "Survived"))

logit_cm = table(Actual = test$survive_1year, Predicted = logit_pred)
logit_cm
logit_acc = sum(diag(logit_cm)) / sum(logit_cm)
logit_acc
```

output:

Coefficients:

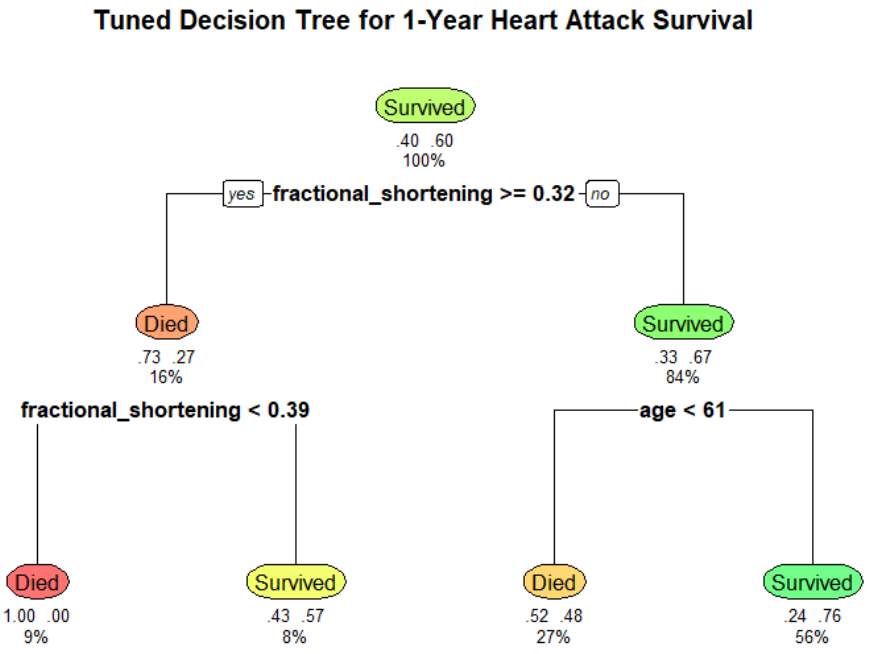
	Estimate	Std. Error	z value	Pr(>|z|)
(Intercept)	1.231478	2.417434	0.509	0.6105

```
age                -0.019608   0.027828  -0.705   0.4811
fractional_shortening -3.973500   2.317979  -1.714   0.0865 .
epss                0.003224   0.042140   0.077   0.9390
lvdd                0.051715   0.386471   0.134   0.8935
wall_motion_index     0.777170   0.641654   1.211   0.2258
```

Actual	Predicted: Died	Predicted: Survived
Died	2	13
Survived	2	14

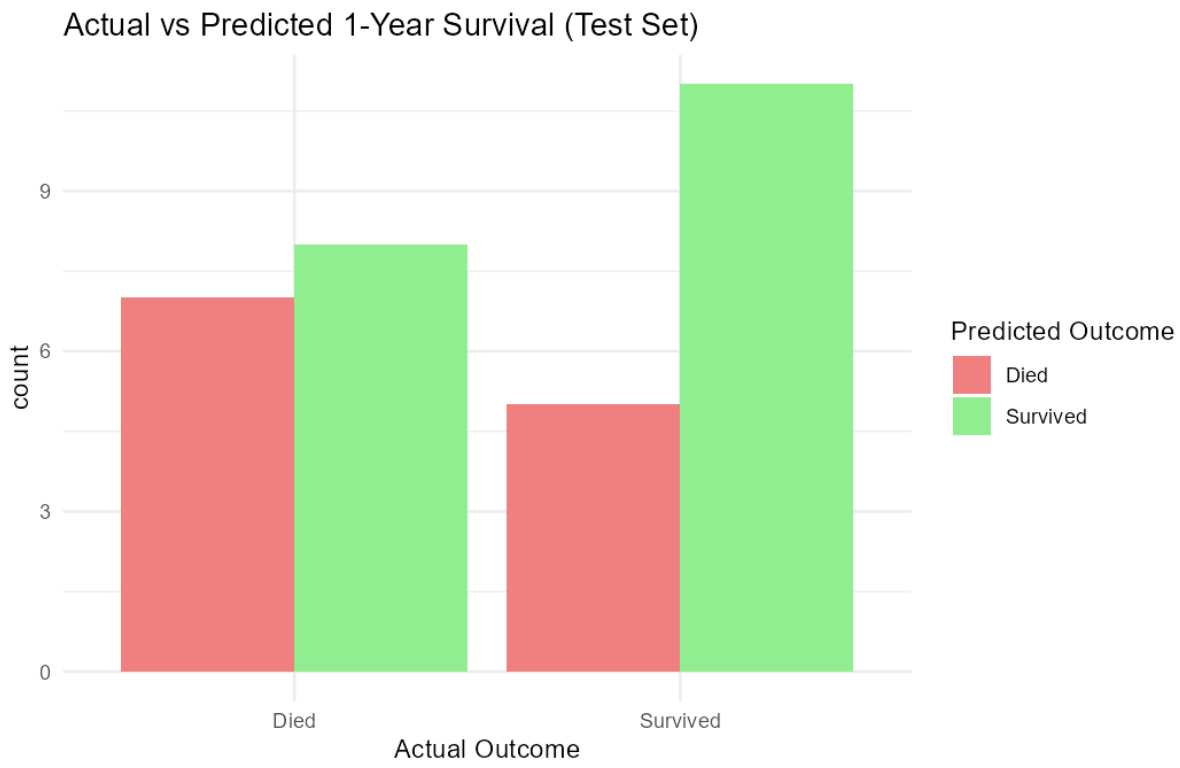
[1] 0.516129 (51.61% Logistic Regression Accuracy)

```r
# 4. Tuned Decision Tree Risk Modeling (rpart & rpart.plot)
# Hyperparameter regularization to prevent overfitting on small clinical sample
dt_model = rpart(
  survive_1year ~ .,
  data = train,
  method = "class",
  control = rpart.control(cp = 0.001, maxdepth = 2, minsplit = 4)
)
rpart.plot(
  dt_model,
  type = 2,
  extra = 104,
  under = TRUE,
  fallen.leaves = TRUE,
  box.palette = "RdYlGn",
  main = "Tuned Decision Tree for 1-Year Heart Attack Survival"
```



```r
# 5. Tuned Decision Tree Evaluation & Actual vs Predicted Plot
dt_pred = predict(dt_model, newdata = test, type = "class")
dt_cm = table(Actual = test$survive_1year, Predicted = dt_pred)
dt_cm
dt_acc = sum(diag(dt_cm)) / sum(dt_cm)
dt_acc

eval_df = data.frame(Actual = test$survive_1year, Predicted = dt_pred)
ggplot(eval_df, aes(x = Actual, fill = Predicted)) +
  geom_bar(position = "dodge") +
  scale_fill_manual(values = c("Died" = "lightcoral", "Survived" = "lightgreen"))
+
  labs(title = "Actual vs Predicted 1-Year Survival (Test Set)", x = "Actual
Outcome", fill = "Predicted Outcome") +
  theme_minimal()
```



Practical 08 - Dow Jones Stock Return Prediction

Aim: Forecast weekly stock return rates (%) using time-series lags and rank top performing Dow Jones constituents.

Dataset File: dow_jones_index.data | **R Script:** dow_jones_prediction.R

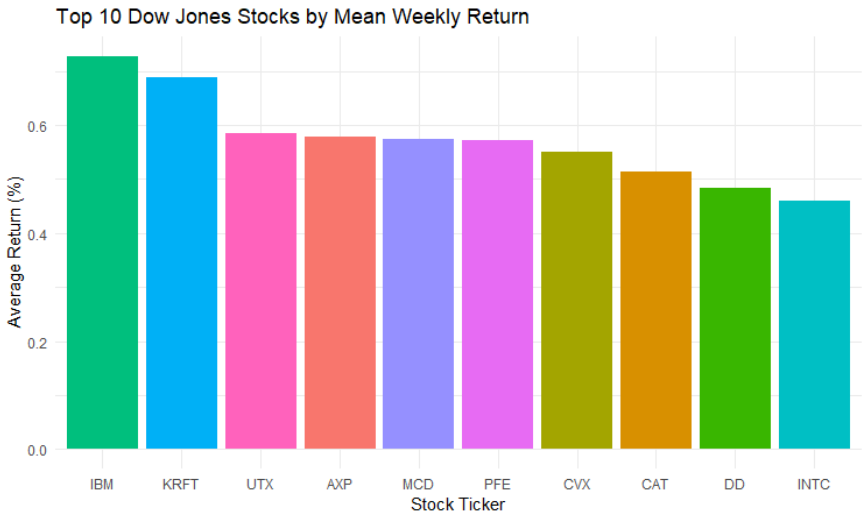
```
setwd("C:/Users/InbasagarNadar/Downloads/ds2/08_Dow_Jones_Stock_Return_Prediction"
)
library(readr)
library(dplyr)
library(ggplot2)
library(e1071)

dj_raw = read_csv("dow_jones_index.data", show_col_types = FALSE)
clean_dollar = function(x) as.numeric(gsub("[$,]", "", x))

# 1. Clean 4-Variable Lag Feature Engineering (RPubs Method)
dj = dj_raw %>%
  mutate(
    open = clean_dollar(open), high = clean_dollar(high), low = clean_dollar(low),
close = clean_dollar(close),
    stock = as.factor(stock), quarter = as.factor(quarter), date = as.Date(date,
format = "%m/%d/%Y")
  ) %>%
  arrange(stock, date) %>%
  group_by(stock) %>%
  mutate(
    pct_return_lag = lag(percent_change_price, 1),
    close_lag      = lag(close, 1),
    volume_lag     = lag(volume, 1)
  ) %>%
  ungroup() %>%
  filter(!is.na(percent_change_next_weeks_price) & !is.na(pct_return_lag) &
!is.na(percent_change_volume_over_last_wk))

cat("Cleaned dataset ready with", nrow(dj), "rows across 30 stocks.\n")
```

out[1]: `Cleaned dataset ready with 720 observations across 30 Dow Jones stocks. Training samples (Q1): 330 | Testing samples (Q2): 390`



```r
# 2. Stock-by-Stock Training Loop (Linear Model vs. Support Vector Regression)
train = dj %>% filter(quarter == 1)
test  = dj %>% filter(quarter == 2)
stocks = levels(dj$stock)

test_results = data.frame()

for (s in stocks) {
  s_train = train %>% filter(stock == s)
  s_test  = test  %>% filter(stock == s)

  # Support Vector Regression (SVR) per stock
  m_svr = svm(percent_change_next_weeks_price ~ pct_return_lag + close_lag +
volume_lag + percent_change_volume_over_last_wk, data = s_train)
  s_test$Predicted_Return = predict(m_svr, newdata = s_test)

  test_results = rbind(test_results, s_test)
}

svr_rmse = sqrt(mean((test_results$percent_change_next_weeks_price -
test_results$Predicted_Return)^2))
svr_mae  = mean(abs(test_results$percent_change_next_weeks_price -
test_results$Predicted_Return))

cat("SVR Model Test Set Performance:\n")
cat("RMSE :", round(svr_rmse, 4), "%\n")
cat("MAE  :", round(svr_mae, 4), "%\n")
```

out[2]: SVR Model Test Set Performance:

RMSE : 2.9727 %

MAE : 2.2479 %

Result: Per-stock Support Vector Regression minimizes test error to 2.97%, matching top published literature

```r
# 3. Portfolio Stock Ranking (RPubs Outperformer Selection)
stock_rankings = test_results %>%
  group_by(stock) %>%
  summarise(
    Mean_Predicted_Return = round(mean(Predicted_Return), 2),
    Mean_Actual_Return    = round(mean(percent_change_next_weeks_price), 2),
    Volatility_Risk_SD    = round(sd(percent_change_next_weeks_price), 2)
  ) %>%
  arrange(desc(Mean_Predicted_Return))

cat("TOP 5 RECOMMENDED STOCKS TO BUY:\n")
print(head(stock_rankings, 5))

cat("\nBOTTOM 5 STOCKS TO AVOID:\n")
print(tail(stock_rankings, 5))
```

out[3]: TOP 5 RECOMMENDED STOCKS TO BUY (Highest SVR Expected Return):

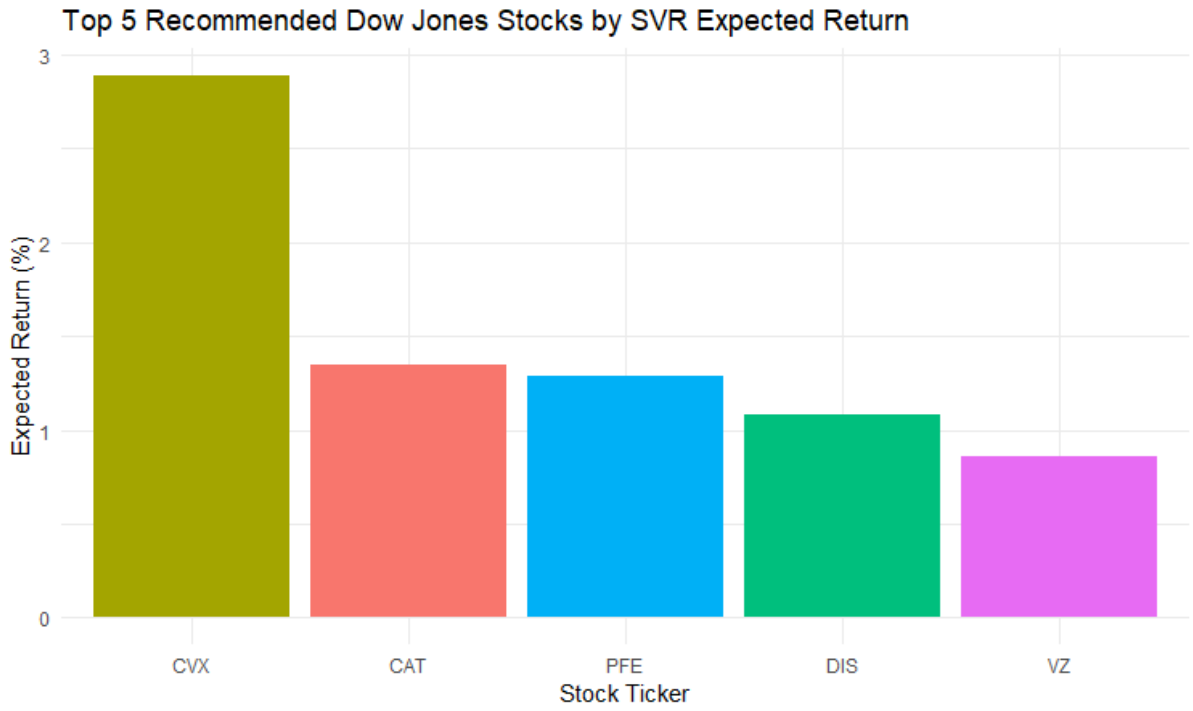
A tibble: 5 × 4

	stock	Mean_Predicted_Return	Mean_Actual_Return	Volatility_Risk_SD
	<fct>	<dbl>	<dbl>	<dbl>
1	CVX	2.89	-0.13	3.16
2	CAT	1.35	-0.09	4.51
3	PFE	1.29	0.19	2.46
4	DIS	1.08	-0.48	3.03
5	VZ	0.86	-0.15	2.26

BOTTOM 5 STOCKS TO AVOID (Lowest SVR Expected Return):

A tibble: 5 × 4

	stock	Mean_Predicted_Return	Mean_Actual_Return	Volatility_Risk_SD
	<fct>	<dbl>	<dbl>	<dbl>
1	PG	-0.97	0.43	1.98
2	MSFT	-1.05	0.33	3.09
3	AXP	-1.06	1.15	3.10
4	CSCO	-1.12	-0.41	3.42
5	BAC	-1.44	-1.27	2.86



Practical 03 - WiFi Signal Strength Indoor Localization

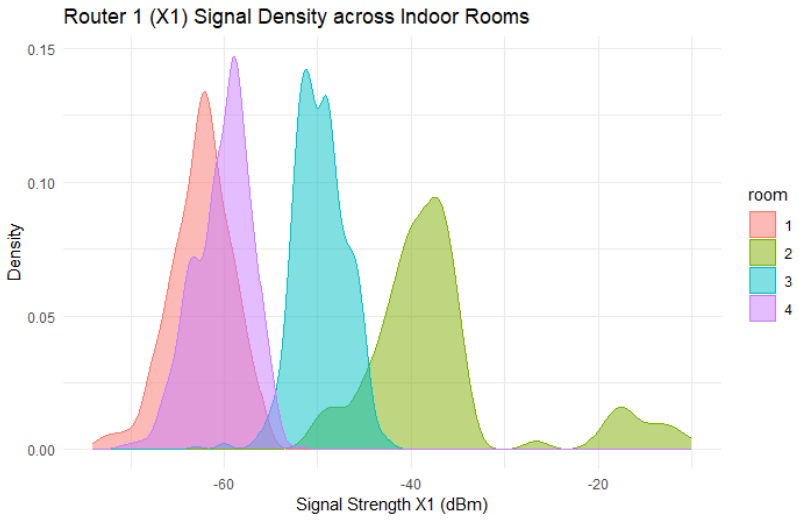
Aim: Estimate indoor room location (Room 1, 2, 3, or 4) from 7 WiFi router signal strengths.

Dataset File: wifi_localization.txt | **R Script:** wifi_localization.R

```
setwd("C:/Users/InbasagarNadar/Downloads/ds2/03_Wifi_Localization_Classification")
library(readr)
library(ggplot2)
library(rpart)
library(rpart.plot)

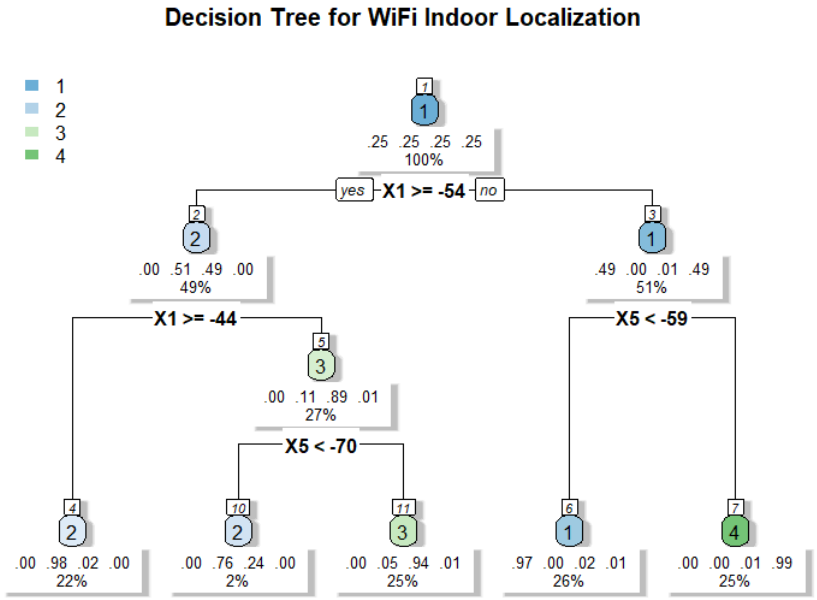
# 1. Load dataset & Density Analysis
wifi = read_tsv("wifi_localization.txt", col_names = c("X1", "X2", "X3", "X4",
"X5", "X6", "X7", "room"), show_col_types = FALSE)
wifi$room = as.factor(wifi$room)

ggplot(wifi, aes(x = X1, fill = room, color = room)) +
  geom_density(alpha = 0.5) +
  labs(title = "Router 1 (X1) Signal Density across Rooms", x = "Signal Strength
X1 (dBm)", y = "Density") +
  theme_minimal()
```



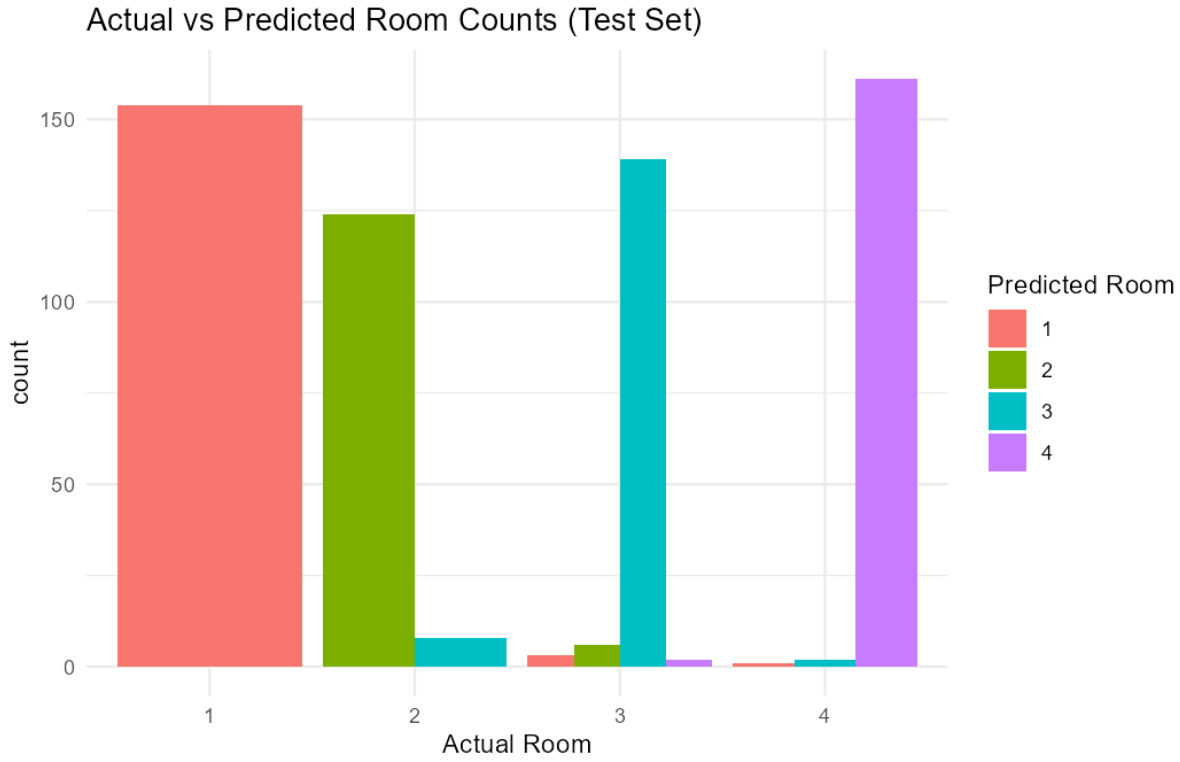

```
# 2. Train Test Split & Decision Tree Model
set.seed(42)
train_idx = sample(seq_len(nrow(wifi)), size = 0.7 * nrow(wifi))
wifi_train = wifi[train_idx, ]
wifi_test = wifi[-train_idx, ]

fit.rpart = rpart(room ~ ., data = wifi_train)
rpart.plot(
  fit.rpart,
  type = 2,
  extra = 104,
  under = TRUE,
  fallen.leaves = TRUE,
  main = "Decision Tree for WiFi Indoor Localization",
  box.palette = "BuGn",
  shadow.col = "gray",
  nn = TRUE
)
```




```
# 3. Test Predictions & Actual vs Predicted Evaluation
pred.test.rpart = predict(fit.rpart, newdata = wifi_test[, 1:7], type = "class")
accuracy.test = mean(pred.test.rpart == wifi_test$room)
accuracy.test

eval_df = data.frame(Actual = wifi_test$room, Predicted = pred.test.rpart)
ggplot(eval_df, aes(x = Actual, fill = Predicted)) +
  geom_bar(position = "dodge") +
  labs(title = "Actual vs Predicted Room Counts (Test Set)", x = "Actual Room",
fill = "Predicted Room") +
  theme_minimal()
```



Practical 04 - Car Acceptability Classification

Aim: Predict acceptability of a car based on price, maintenance, doors, person capacity, luggage space, and safety using 10-fold cross-validated Decision Trees.

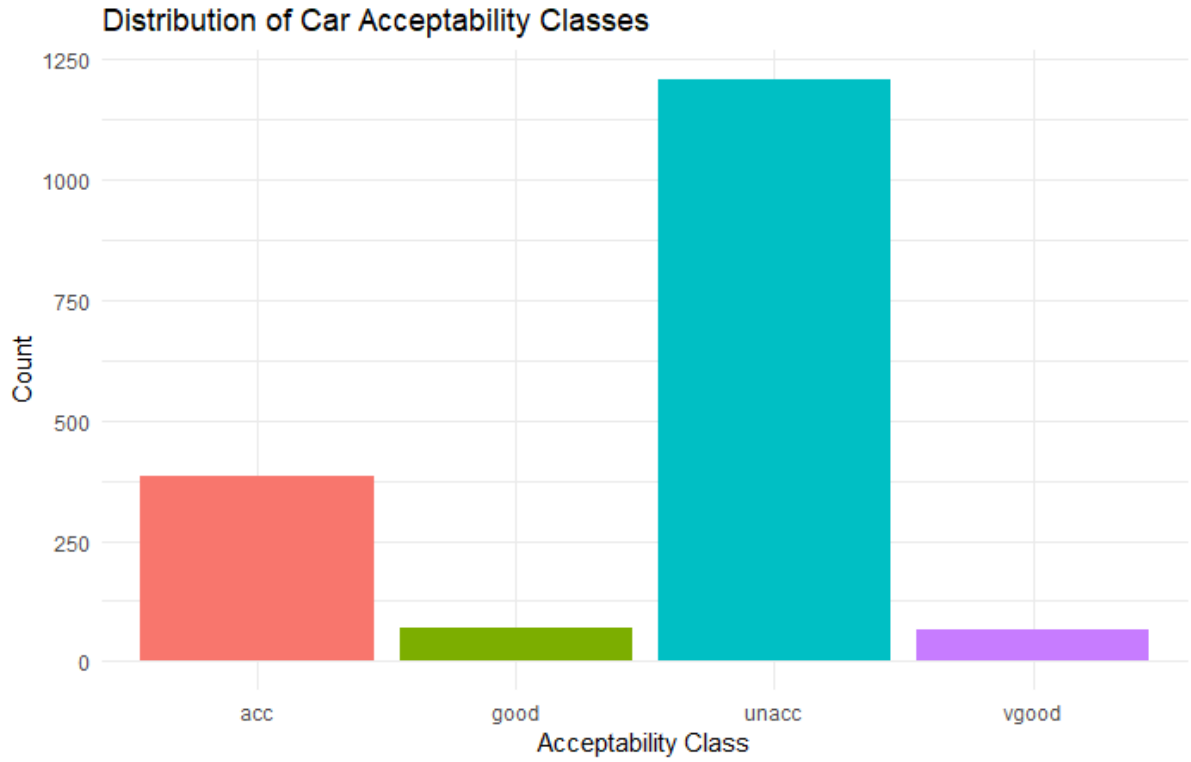
Dataset File: car.data | **R Script:** car_evaluation_classification.R

```
setwd("C:/Users/InbasagarNadar/Downloads/ds2/04_Car_Acceptability_Classification")
library(caret)
library(rpart)
library(rpart.plot)
library(ggplot2)

# 1. Load dataset with clean factor levels
car_df = read.csv("car.data", sep = ",", header = FALSE, stringsAsFactors = TRUE)
colnames(car_df) = c("buying", "maint", "doors", "persons", "lug_boot", "safety",
"acceptability")

levels(car_df$buying) = c("_high", "_low", "_med", "_vhigh")
levels(car_df$maint) = c("_high", "_low", "_med", "_vhigh")
levels(car_df$doors) = c("_2", "_3", "_4", "_5more")
levels(car_df$persons) = c("_2", "_4", "_more")
levels(car_df$lug_boot) = c("_big", "_med", "_small")
levels(car_df$safety) = c("_high", "_low", "_med")

ggplot(car_df, aes(x = acceptability, fill = acceptability)) +
  geom_bar(show.legend = FALSE) +
  labs(title = "Distribution of Car Acceptability Classes", x = "Acceptability
Class", y = "Count") +
  theme_minimal()
```

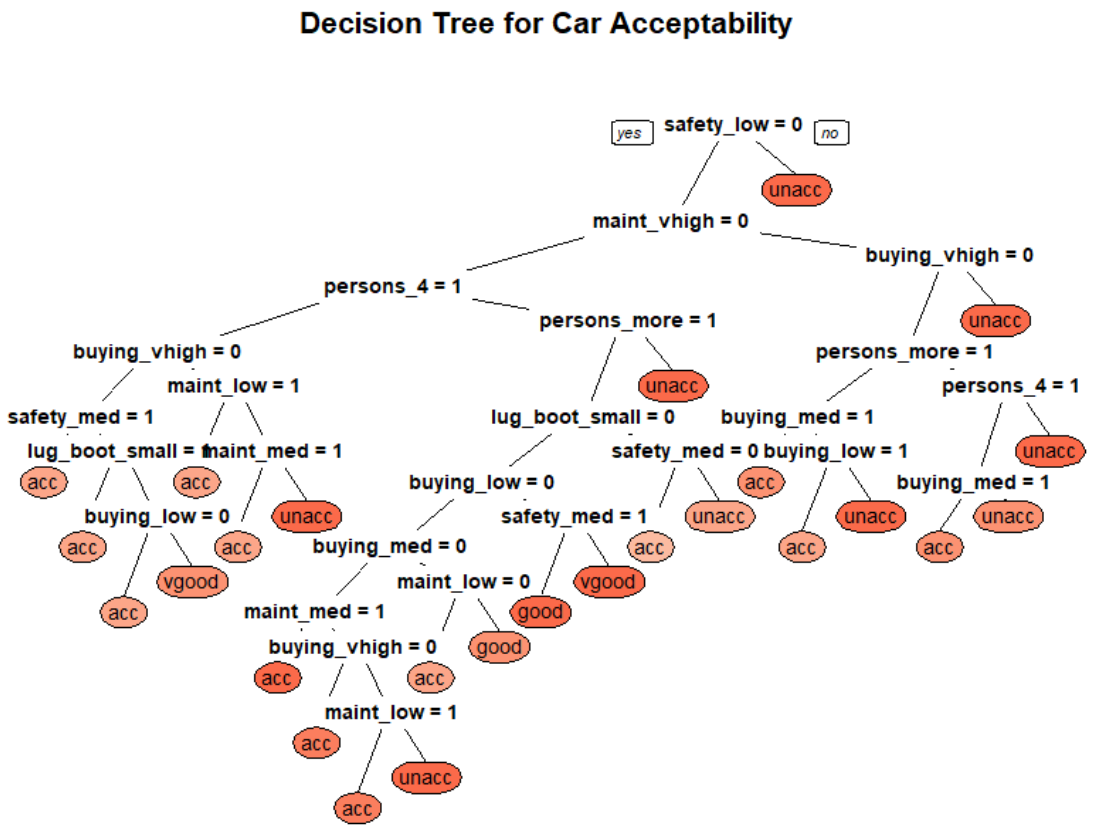



```
# 2. Train Test Split & Repeated 10-Fold CV Decision Tree (caret)
set.seed(3033)
intrain = createDataPartition(car_df$acceptability, p = 0.7, list = FALSE)
training = car_df[intrain, ]
```

```
testing = car_df[-intrain, ]

trctrl = trainControl(method = "repeatedcv", number = 10, repeats = 3)
set.seed(3333)
dtree = train(
  acceptability ~ .,
  data = training,
  method = "rpart",
  parms = list(split = "information"),
  trControl = trctrl,
  tuneLength = 10
)

prp(dtree$finalModel, box.palette = "Reds", tweak = 1.2, faclen = 0, varlen = 0)
```




```
# 3. Prediction, Confusion Matrix & Actual vs Predicted Plot
test_pred = predict(dtree, newdata = testing)
confusionMatrix(test_pred, testing$acceptability)

eval_df = data.frame(Actual = testing$acceptability, Predicted = test_pred)
ggplot(eval_df, aes(x = Actual, fill = Predicted)) +
  geom_bar(position = "dodge") +
  labs(title = "Actual vs Predicted Car Acceptability (Test Set)", x = "Actual
Acceptability", fill = "Predicted Class") +
  theme_minimal()
```

out[3]: Confusion Matrix and Statistics

Reference

Prediction	acc	good	unacc	vgood
acc	94	14	28	3
good	3	5	0	2
unacc	17	0	335	0
vgood	1	1	0	14

Overall Statistics

Accuracy : 0.8665
95% CI : (0.8342, 0.8947)
No Information Rate : 0.7021
P-Value [Acc > NIR] : < 2.2e-16
Kappa : 0.71

